FORGE / APPFORGE

AI Studio-Ready Product Requirements

A traceable specification built from the complete conversation source

SOURCE REVIEWED

Pasted text.txt - 2,495 lines / 89,339 characters

STATUS NOTE

The source mixes assistant proposals, examples, code, and brief user replies. Candidate features are labeled accordingly; no unresolved choice is silently treated as confirmed.

Prepared 12 November 2026

Contents

Instructions to the AI coding assistant
1. Evidence and decision status
2. Product summary and intended users
3. Candidate product requirements
4. Proposed technical direction (not confirmed)
5. Traceability checklist
6. Open questions and source inconsistencies
7. Implementation and verification guidance
8. Source coverage map

Instructions to the AI coding assistant

Use every item in this brief. Do not omit, simplify, silently reinterpret, or claim completion of any requirement. Preserve the distinction between sourced product intent, proposed implementation, illustrative examples, and unresolved choices. Do not promote a suggestion or sample into a confirmed product decision. Before implementation, inspect the existing repository and explain conflicts with its current architecture. Where this brief identifies an unresolved choice, either ask the user or isolate the decision behind a reversible default and clearly report it; do not invent product behavior. Implement real functionality rather than visual-only controls. Validate every generated, imported, saved, exported, or published project against one agreed schema. Never claim a live integration or successful publish unless it was actually exercised.

1. Evidence and decision status

The only unambiguous user-authored intent visible in the referenced chat is the request for a PDF that captures requirements without losing any. The attachment itself frames Forge/AppForge as a drag-and-drop app and website builder with an AI Studio. Most detailed features and implementation decisions were written by the assistant in the source conversation. The user's repeated "yes" replies indicate interest, but several followed prompts offering multiple alternatives; the assistant sometimes selected a scope on the user's behalf. Therefore:

- **User goal (confirmed):** Produce a complete, usable Forge/AppForge requirements brief from the supplied conversation content.
- **Product concept (source-stated, not independently user-authored in the visible record):** A visual, drag-and-drop builder for apps and websites, assisted by AI.
- **Candidate requirements (CR):** Capabilities repeatedly described as part of the proposed Forge product. They are retained for review and implementation planning, but are not all proven to be explicit user decisions.
- **Implementation proposals (IP):** Specific stacks, libraries, data structures, file names, and approaches suggested in the source. They are not binding unless the user confirms them.
- **Examples (EX):** Sample text, colors, templates, schema values, routes, code snippets, and marketing claims. These illustrate ideas; they are not production content or guaranteed behavior.
- **Unresolved choices (UQ):** Ambiguities, conflicts, missing scope, and implementation gaps that must not be silently decided as requirements.

The traceability checklist uses **Review status: Confirmed goal** only for the user's document objective. All Forge product rows are marked **Source-derived candidate** unless explicitly identified as an example or unresolved choice. This is intentional: short "yes" answers do not establish which item was selected when the preceding question offered multiple choices.

2. Product summary and intended users

Forge (also called AppForge in the source) is proposed as a visual studio in which users compose web pages and app screens from editable components, use AI to create or revise layouts, preview interactions, and export or publish projects. The source positions it for beginners and developers: beginners get visual editing, templates, and an in-product backend; developers get editable structured data and readable code export. These user segments and value claims are source framing, not validated market decisions.

The product is expected to support both website pages and multi-screen app experiences. The source repeatedly describes functional output rather than static mockups, but the actual backend, authentication, data, runtime, security, and deployment scope is not specified in enough detail to treat as settled.

3. Candidate product requirements

3.1 Studio workspace and editing

- **CR-UI-01 - Three-column studio:** Provide a left sidebar for assets/components/structure, a central live canvas, and a right properties panel. (Source lines 4-10, 904-906)
- **CR-UI-02 - Component library:** Group draggable blocks and pre-made section templates. The explicit example categories are Layout, Basic, and Forms. (7, 779-888, 964-990)

- **CR-UI-03 - Canvas editing:** Render a responsive, live workspace where users can drop components. The opening blueprint calls for alignment guides, snap-to-grid, and device toggles. (8)
- **CR-UI-04 - Nested placement:** Allow dropping onto the page root and inside eligible Container, Card, or Navbar nodes; preserve parent-child relationships. (996-1025)
- **CR-UI-05 - Click-to-add:** Clicking a component in the sidebar should add it to the selected container or root, as an alternative to drag-and-drop. (1022-1026)
- **CR-UI-06 - Selection and deselection:** Canvas clicks select a node for editing; clicking outside the page content clears selection. (268-314, 767-771)
- **CR-UI-07 - Properties panel:** Show context-aware editable content, layout, appearance, and actions for the selected node; show a helpful empty-selection message otherwise. (396-401, 423-680)
- **CR-UI-08 - Supported property controls in the example:** Text for Heading/Text; level for Heading; label for Button; placeholder for Input; source URL for Image; direction for Container/Card/Navbar; padding, gap, width, radius, background, text color, font size, text alignment; and Button click action/target. (472-680)
- **CR-UI-09 - Immediate updates:** Property edits should update the canvas immediately. (767-771)
- **CR-UI-10 - Delete and descendants:** Allow deleting a selected node. The example cascades deletion to direct children; intended behavior for deeper descendants must be clarified/implemented safely. (464-469, 734-737, 769-771)
- **CR-UI-11 - Top action bar:** Include project name, Undo/Redo, Live Preview, AI prompt access, and a primary Export/Publish action. (10, 1321-1404)
- **CR-UI-12 - Visual polish:** Keep the workspace familiar and uncluttered, with a clear place for each tool. This is a qualitative design requirement without supplied acceptance metrics. (6)

3.2 AI Studio

- **CR-AI-01 - Prompt-to-UI:** Accept a plain-language prompt and generate a page, section, or component as structured, editable Forge nodes on the canvas. The example asks for a three-tier pricing section. (15, 904-962, 1026)
- **CR-AI-02 - Generation modes:** Support generating incrementally and either appending generated nodes or replacing the current page; the source presents both as options, not a resolved default. (1177-1191)
- **CR-AI-03 - Contextual editing:** Offer an AI action bar for a selected element, with requests such as improving copy tone or changing a card to dark mode. (17)
- **CR-AI-04 - Image-to-UI:** Accept wireframes, sketches, or screenshots and reconstruct editable nodes, layout hierarchy, colors, and text, preserving row/column groupings. (16, 2404-2481)
- **CR-AI-05 - Upload formats in example:** PNG, JPEG/JPG, and WebP are named across the source. (2410-2416, 2472-2479)
- **IP-AI-01 — Upload processing example:** The sample browser flow reads the selected file as a data URL, removes the prefix, and sends base64 data plus MIME type to the image-generation call. This is an implementation example; upload limits, storage, privacy, and error handling are unresolved. (2455–2469)
- **CR-AI-06 - Smart data/logic assistance:** AI may connect forms to data sources, generate sample content, and set up basic navigation. These are high-level capabilities; exact supported integrations and safe automation rules remain unspecified. (18)
- **CR-AI-07 - Structured output:** AI output should conform to a strict schema so the canvas can render it predictably. The source proposes a bounded flat `nodes` array with `parentId` links for Gemini structured output. (59-67, 162-257)
- **CR-AI-08 - Validation and failures:** Reject or handle empty, malformed, unsupported, or structurally invalid AI results without corrupting the project. The code sample handles empty response and parse/API errors, but does not fully specify validation or user recovery. (1123-1164, 1171-1185)
- **CR-AI-09 - Unique IDs:** Generated nodes must have collision-free IDs, and parent references must be remapped consistently before insertion. (149, 1151-1163, 1190)
- **CR-AI-10 - Prompting constraints in example:** Generate clean, modern, well-spaced layouts from Container, Navbar, Card, Heading, Text, Button, Input, and Image; top-level nodes use `parentId: "root"`; children reference eligible

parent IDs; use hex colors, generous spacing, and valid placeholder image URLs. (1123-1132)

3.3 Components and project data

- **CR-DATA-01 - Project data model:** Store a project identifier/name, platform, theme, and pages/screens. The sample theme includes primary, background, and text colors, font family, and border radius. (62-83)
- **CR-DATA-02 - Page model:** Each page has a unique ID, display name, route, and component nodes. The later project proposal adds `activePageId`. (84-101, 1419-1435)
- **CR-DATA-03 - Node model:** Each UI node has a unique ID, component type, editable props, styles, events, and children or parent linkage. (62-67)
- **CR-DATA-04 - Node types:** The explicit registry is Container, Navbar, Heading, Text, Button, Image, Input, and Card. Unknown node types should not crash rendering; the code examples return no output for unknown types, but user-facing behavior is unresolved. (142, 202-213, 315-384)
- **CR-DATA-05 - Separation of content and styling:** Keep component props separate from style values. (143)
- **CR-DATA-06 - Event representation:** Model simple interactions, including navigation, form submission, and modal opening. Sample actions are `NONE`, `NAVIGATE`, `SUBMIT_FORM`, and `OPEN_MODAL`, with a target string. (126-131, 241-249)
- **CR-DATA-07 - Validation/versioning:** Use unique IDs; consider normalized `nodesById`/`childIds` for large projects; include a schema version (sample `1.0`) to support upgrades. Unique IDs/versioning are recommendations in the source; normalized storage is explicitly optional. (147-151)
- **CR-DATA-08 - Serializable/editable state:** Keep project state structured so the canvas, AI generator, live preview, and exporters share one source of truth. (59-60)

3.4 Pages, templates, persistence, history, and layers

- **CR-PROJ-01 - Multi-page support:** Users can switch between multiple pages/screens, with shared headers, footers, and navigation bars. (24, 1417-1421)
- **CR-PROJ-02 - Page management:** Show page tabs with name and route; create pages from a name and derive a route in the example. Page deletion, rename, duplicate, route conflicts, and validation are not specified. (1579-1686)
- **CR-PROJ-03 - Starter templates:** Offer categorized templates, with the source explicitly showing SaaS landing plus login, a portfolio as an example category, and a blank canvas. E-commerce storefronts and dashboards are also named as possible categories. (23, 1440-1576, 1664-1682)
- **CR-PROJ-04 - Local autosave and reload:** Persist projects in browser LocalStorage and restore them after refresh; recover from corrupt saved JSON by falling back to a starter. (1689-1712)
- **CR-PROJ-05 - Undo/Redo:** Track project state with undo and redo stacks, avoid recording identical states, retain a maximum of 30 undo steps, and clear redo history after a new edit. The examples cover generation, drag/drop, and property edits. (1742-1818)
- **CR-PROJ-06 - Keyboard shortcuts:** Support Ctrl/Cmd+Z for undo, Ctrl/Cmd+Shift+Z for redo, and Ctrl/Cmd+Y for redo, while not intercepting active inputs or textareas. (1821-1845)
- **CR-PROJ-07 - Layers tree:** Show a recursive hierarchy in the left sidebar, with one-click selection of nested elements, node type, a short content preview, and Move Up/Move Down controls for siblings. (1848-1949)
- **CR-PROJ-08 - Reordering:** Move a node among siblings without changing parent relationships or detaching its nested children. (1952-1987)

3.5 Preview, export, and publishing

- **CR-OUT-01 - Device preview:** Provide desktop and mobile view toggles; the sample mobile width is 390 px. The opening blueprint also names tablet, which is absent from the later toolbar example. (8, 1321-1379)
- **CR-OUT-02 - Interactive live preview:** Hide editor sidebars/outlines and let users test inputs, buttons, and navigation as an end user. (1321-1323, 1407-1411)
- **CR-OUT-03 — HTML/CSS/JavaScript ZIP export:** Export a standalone HTML experience with its styling and working interactions, plus a project JSON backup in a downloadable ZIP; use a safe project-derived filename. The opening brief

names HTML/CSS/JS as a target format; the later sample compiler primarily emits HTML with inline styles and limited navigation. (26, 1200–1318)

- **CR-OUT-04 - Multi-page ZIP:** Include every project page in one archive, mapping `/` to `index.html` and examples such as `/login` to `login.html`; include a full project JSON backup. (1689-1735)
- **CR-OUT-05 - React export:** Also provide readable JSX/React output in the ZIP; the later proposal packages HTML and React separately (e.g., `html/index.html`, `react/Home.jsx`). (1994-2075, 2178-2181)
- **CR-OUT-06 - Output quality:** Source describes exported output as clean, semantic, production-ready, readable, modular, and usable without vendor lock-in. Treat these as quality goals requiring verification, not claims already met by the snippets. (26, 1207-1209, 1996-1998, 2390-2395)
- **CR-OUT-07 - Publish and share:** Publish project JSON to a cloud service and return a live shareable URL, with a public viewer that loads a published project and supports multi-page navigation. (2078-2175)
- **CR-OUT-08 - Publish controls:** Include a Publish Live action and copy the resulting viewer URL to the clipboard. (2178-2181)
- **CR-OUT-09 - Hosting examples:** Exported standalone HTML is intended to be hostable on Vercel, Netlify, or GitHub Pages; these are examples, not selected hosting providers. (1409-1411)

3.6 Backend, data, and integration concepts

- **CR-BE-01 - Built-in backend concept:** Provide visual data tables, user-authentication toggles, and form-submission handling without requiring external setup. This is stated at concept level only. (25)
- **CR-BE-02 - Form/data wiring:** AI-assisted connections between forms and databases and sample content are proposed. Exact providers, data model, CRUD operations, validation, authorization, and deployment behavior are open. (18)
- **CR-BE-03 - Supabase option:** The implementation proposal uses a `forge_projects` table with text ID, app name, JSONB project data, and update timestamp; upsert publishes and fetch reads by project ID. This is a suggested implementation, not a confirmed provider. (2078-2137)
- **CR-BE-04 - API key handling:** The source explicitly warns to keep Gemini keys on a backend/API route in production rather than exposing them in client code or network logs. (1048-1049, 1188-1192)

3.7 Marketing page and illustrative content

The attachment includes a sample Forge public landing page with a navbar, “Open Studio” call to action, hero heading/subtext, “Start Building Free” button, and three feature cards for drag-and-drop, AI Studio, and code export. It also includes sample SaaS and login template copy, example colors, routes, IDs, and placeholder URLs. These are **examples**, not confirmed branding, final marketing claims, pricing, account features, or production copy. (1445-1576, 2187-2401)

4. Proposed technical direction (not confirmed)

The source recommends considering:

- Craft.js, Puck, or GrapesJS for drag-and-drop/canvas state and serialization.
- React, Next.js, and Tailwind CSS for the editor UI and generated output.
- Google AI Studio / Gemini API for text and image-to-layout generation with structured JSON output; the sample names `@google/genai` and `gemini-2.5-flash`.
- Electron or Tauri if a Windows/Mac desktop wrapper is needed.
- JSZip for browser ZIP packaging.
- Supabase for cloud project storage and live sharing; Firebase is mentioned only as an alternative in a later prompt, not implemented.
- Browser LocalStorage for an initial autosave implementation.
- Native HTML5 drag-and-drop as a zero-dependency implementation option.

The suggested repository blueprint names these files and responsibilities: `src/types/forgeProjectTypes.ts` for project/page/node interfaces; `src/data/forgeBlueprints.ts` for component presets and starter templates; `src/hooks/useForgeHistory.ts` for 30-step history; `src/services/forgeAIService.ts` for Gemini text and image-to-UI; `src/services/forgeExporter.ts` for HTML, React JSX, and multi-page ZIP; `src/services/forgeCloudService.ts` for Supabase publish/viewer sync; and `src/components/ForgeStudio.tsx` for the top bar, page switcher, left AI/blocks/layers sidebar, canvas, and properties panel. These paths are proposals, not requirements to create those exact files. (2484–2493)

These are **implementation suggestions from the source**, not user-confirmed decisions and not verified for current compatibility, pricing, security, or production suitability. An implementing assistant should check the repository and current official documentation before selecting or installing them. Do not use a client-exposed secret. Do not assume the source code samples are complete or safe to deploy as-is.

5. Traceability checklist

Status key: **[ ] Source-derived candidate** means retain and review; it does not assert explicit user approval. **[?] Unresolved** means the assistant must not silently decide. A checked box should only be marked complete after implementation and verification evidence exist.

ID	Requirement / decision to track	Source lines	Status
UI-01	Three-column studio layout	4-10	[ ] Source-derived candidate
UI-02	Component/section library with categories	7, 779-888	[ ] Source-derived candidate
UI-03	Responsive canvas, guides, grid, device toggles	8	[ ] Source-derived candidate
UI-04	Nested drag/drop placement	996-1025	[ ] Source-derived candidate
UI-05	Click-to-add into selected container/root	1022-1026	[ ] Source-derived candidate
UI-06	Selection, outside-click deselection, immediate edits	268-314, 767-771	[ ] Source-derived candidate
UI-07	Context-aware properties listed in CR-UI-08	396-680	[ ] Source-derived candidate
UI-08	Delete behavior including full descendant handling	464-469, 734-737	[?] Descendant depth unclear
UI-09	Top bar: name, undo/redo, preview, AI, export/publish	10, 1321-1404	[ ] Source-derived candidate
AI-01	Prompt creates editable nodes	15, 904-962	[ ] Source-derived candidate
AI-02	Append vs replace generation behavior	1177-1191	[?] Default/mode unresolved
AI-03	Contextual selected-element assistant	17	[ ] Source-derived candidate
AI-04	Screenshot/sketch upload to editable UI	16, 2404-2481	[ ] Source-derived candidate
AI-05	Image formats PNG/JPEG/WebP	2410-2416, 2472-2479	[ ] Source-derived candidate
AI-06	AI-assisted data wiring/navigation/sample data	18	[?] Scope/providers unresolved
AI-07	Strict schema validation and collision-proof IDs	162-257, 1151-1163	[ ] Source-derived candidate
DATA-01	Shared project/page/node schema	59-101, 1419-1435	[ ] Source-derived candidate
DATA-02	Component registry of eight named types	202-213	[ ] Source-derived candidate
DATA-03	Props/styles/events separation	142-145, 215-249	[ ] Source-derived candidate
DATA-04	Schema versioning and unique IDs	149-151	[ ] Source-derived candidate
PROJ-01	Multi-page route/page switching	24, 1579-1686	[ ] Source-derived candidate
PROJ-02	Shared headers/footers/navigation	24	[?] Reuse mechanism unresolved
PROJ-03	Starter templates: SaaS, portfolio, shop, dashboard, blank	23, 1440-1576	[?] Initial catalog unresolved
PROJ-04	Local autosave, reload, corrupt-data fallback	1689-1712	[ ] Source-derived candidate
PROJ-05	30-step undo/redo and keyboard shortcuts	1742-1845	[ ] Source-derived candidate
PROJ-06	Recursive layers tree with sibling reorder	1848-1987	[ ] Source-derived candidate
OUT-01	Desktop/mobile preview; tablet also mentioned	8, 1321-1379	[?] Device targets unresolved
OUT-02	Interactive preview behavior for all event types	1407-1411	[?] Snippets only implement some interactions
OUT-03	HTML ZIP with JSON backup	1200-1318	[ ] Source-derived candidate
OUT-04	Multi-page archive and route-to-file mapping	1689-1735	[?] Nested/duplicate routes unresolved

ID	Requirement / decision to track	Source lines	Status
OUT-05	React JSX export alongside HTML	1994-2075, 2180-2181	[ ] Source-derived candidate
OUT-06	Production-ready/semantic/export quality	26, 1207-1209	[?] Acceptance criteria required
CLOUD-01	Cloud publish, public viewer, share URL	2078-2175	[?] Provider/access model unresolved
CLOUD-02	Copy live URL after publish	2178-2181	[ ] Source-derived candidate
BE-01	Built-in data tables/auth/form handling	25	[?] Functional scope unresolved
BE-02	Gemini key stays server-side in production	1048-1049, 1188-1192	[ ] Source implementation constraint
TECH-01	Select canvas/framework/AI/export/cloud stack	28-36, 1034-1045, 1200-1205, 2078-2104	[?] Suggestions only
BRAND-01	Sample public Forge marketing landing page	2187-2401	[ ] Example only
DOC-01	Preserve source distinctions and do not omit/simplify	This brief	Confirmed goal

6. Open questions and source inconsistencies

These are questions exposed by the source, not new requirements. Resolve them with the product owner or document explicit decisions before implementation.

1. **Which capabilities are actually in scope for the first release?** The source accumulates a broad builder, AI features, backend tools, templates, multiple export formats, cloud publishing, and a marketing site without prioritization.

2. **Which schema is canonical?** Early content uses a recursive `root` object with `children`; the structured-output design uses a flat `nodes` array with `parentId`; later page objects use `nodes`; the first project schema includes platform/theme/schemaVersion while the later `ForgeProject` interface omits them. The Gemini API schema sample also differs from the earlier JSON schema (theme absent in the later typed schema). Choose one versioned model and migration policy.

3. **What does each event do at runtime?** The action list includes form submit and modal open, but sample preview/export logic primarily handles navigation. Define behavior and test it across editor preview, HTML export, React export, and published viewer.

4. **What is “built-in backend”?** Data tables, authentication toggles, form submission, and database wiring are named but no auth provider, access rules, data operations, validation, or server behavior is specified.

5. **What is the cloud publishing security model?** The Supabase example uses a public anon key and omits row-level security, ownership, edit permissions, private projects, abuse protection, deletion/unpublish, and project ID allocation. Decide whether projects are public or private and define access controls before implementation.

6. **What counts as production-ready export?** The sample string compilers do not establish escaping, accessibility, runtime dependencies, event completeness, responsive CSS, test/build setup, nested route correctness, or safe handling of user-provided text/URLs. Define measurable export acceptance criteria.

7. **Which devices and breakpoints are supported?** Tablet appears in the concept, but only desktop/mobile appear in the example toolbar; responsive behavior is not fully specified.

8. **How do page routes map to files and navigation?** The sample mapping may not handle nested paths, collisions, query strings, or client-side routing. Define route normalization and navigation behavior.

9. **Which generation insertion mode is expected?** The source says append or replace and demonstrates append. Decide whether users choose per generation or there is a stable default.

10. **What limits apply to image uploads and generated content?** The source does not set file size/dimension limits, user consent/data retention, unsupported formats behavior, AI cost/usage controls, or privacy details.

11. **What is the approved template and marketing content?** Product name (Forge vs AppForge), copy, visual identity, signup/free-plan claims, and template catalog need owner approval; sample strings are not commitments.

12. **Are desktop apps required?** Electron/Tauri are optional suggestions only; the product is otherwise described as a web studio.

13. **What does “shared headers, footers” mean?** There is no reusable-component mechanism defined; determine whether these are duplicated nodes, linked symbols, or layout inheritance.

7. Implementation and verification guidance

Treat the requirements as testable behavior, not just visible controls. At minimum, verification should demonstrate: valid project round-trip through save/load; nested selection/edit/delete/reorder without orphan nodes; stable undo/redo including AI and drag edits; prompt and image output validation with recoverable errors; preview actions matching their declared semantics; complete multi-page HTML/React exports with preserved routes and project JSON; and a publish/read flow protected by the chosen access model. Verify with real API/service configuration only when available, and report clearly which checks were performed. The source does not provide a release schedule, target platform decision, performance budget, accessibility standard, user account model, or success metrics; do not invent these.

8. Source coverage map

This brief reviewed and represented the full attachment, not just its opening preview:

- Lines 1-56: product concept, initial workspace, AI capabilities, general features, stack suggestions, search-result text.
- Lines 57-160: project JSON architecture/sample, rationale, implementation notes, and a user “yes” followed by assistant-selected scope.
- Lines 162-391: strict flat JSON schema, AI Studio usage instructions, canvas renderer.
- Lines 394-772: properties controls, editor wiring, selection, deletion, live updates.
- Lines 775-1028: component presets, AI/sidebar, drag/drop and click-to-add behavior.
- Lines 1030-1194: Gemini structured-output proposal, generation workflow, ID remapping, key-security warning.
- Lines 1196-1413: HTML/ZIP export, top bar, preview and device-mode behavior.
- Lines 1415-1738: multi-page data model, SaaS/blank templates, page bar, LocalStorage, multi-page ZIP.
- Lines 1740-1992: history, keyboard shortcuts, layers tree and sibling reordering.
- Lines 1992-2185: React exporter, Supabase publish/fetch, public viewer and top-bar actions.
- Lines 2185-2495: example Forge marketing landing page, image-to-UI upload, folder blueprint.

The source also includes browser search result excerpts at lines 42-55. They are incidental external snippets, not Forge requirements, and have not been converted into product claims.